Animare la wall slide

Stati di animazione e layer disaccoppiati dall'ABP

Ramo `claude/project-thread-2vuw3f` (da `claude/project-thread-v1plhd`)

25 ottobre 2026

Indice

1 Cosa c'è in questo documento **1**

2 Stati della wall slide **1**
2.1 Ragionamento . . . 1
2.2 Pseudocodice . . . 2
2.3 Classi e strumenti Unreal . . . 2
2.4 Il grafo del layer (con animazioni segnaposto) . . . 2

3 Animazioni per funzionalità, disaccoppiate dall'ABP **3**
3.1 Ragionamento . . . 3
3.2 Parte A (fatta) . . . 3
3.3 Parte B (fatta): il plugin GSTNAnimation . . . 3
3.4 La hitbox non cambia: le regole . . . 3

4 Le tue scelte e gli esempi **3**
4.1 Classi del plugin . . . 4

5 Cosa controllare a mano **4**

1 Cosa c'è in questo documento

La wall slide v2 (`WallSlidingV2`, `WallClimbingV2`) ora espone all'animazione tutto ciò che serve per più stati: entrata, uscita, scivolata da fermo, verso sinistra, verso destra e skid sopra una certa velocità. In più c'è il primo pezzo dello schema generale che hai chiesto: le funzionalità portano le loro animazioni in *layer* separati, scelti da dati, e l'ABP principale non le nomina mai, e il plugin `GSTNAnimation` che sceglie le varianti per skin con regole a tag (la tua scelta, sezione 4), con hitbox identiche.

La parte A è stata compilata sul fisso (905/905 test verdi). Il plugin (parte B) è scritto nel cloud e **va ancora compilato** (sezione 5). Le modifiche all'ABP non sono fatte: l'ABP è un Blueprint condiviso e l'editor del fisso è in uso dall'altro thread.

2 Stati della wall slide

2.1 Ragionamento

L'animazione non fa parte della simulazione: si ricava a ogni frame, sul game thread, dallo stato di presentazione di Mover (lo stato sincronizzato già smussato sui proxy). Per questo può tenere i suoi timer (durata dell'entrata e dell'uscita) senza toccare predizione e rollback, e le modalità di movimento non sanno nulla dell'animazione.

"Sul muro" significa: la modalità attiva è della classe `UGSTN_WallTraverseModeBase` (slide e climb v2). Così il passaggio slide ↔ climb sul posto non è un'uscita seguita da un'entrata. Le modalità legacy sono ignorate, come hai chiesto, ma basta cambiare la classe nelle impostazioni per includerle.

La velocità viene scomposta sul piano del muro: *laterale* (positiva verso la destra del personaggio, che guarda il muro), *verticale* (lungo la gravità proiettata sul muro, positiva in su; funziona con qualsiasi direzione della gravità) e *velocità sul piano*. Lo skid è una regola di velocità perché il freno non è nello stato sincronizzato: un'entrata veloce frena e passa attraverso lo skid in modo naturale. Ogni soglia ha un'isteresi, così lo stato non sfarfalla al confine.

2.2 Pseudocodice

```
ogni frame (game thread, prima del grafo):
  ingressi = modalità attiva, velocità, normale filtrata del muro, gravità
  fase:
    None/Exit e sul muro -> Enter (o Active se EnterDuration = 0)
    Enter da EnterDuration -> Active
    Enter/Active e non sul muro -> Exit (salva la modalità che subentra)
    Exit da ExitDuration -> None
  se sul muro:
    (laterale, verticale, velocità) = scomponi(velocità, normale, gravità)
    se non climb e velocità >= (skid prima ? 650 : 750) -> Skid
    altrimenti se |laterale| >= (stesso lato prima ? 80 : 120) -> SlideLeft / SlideRight
    altrimenti -> Idle
  fuori dal muro: stato e velocità restano gli ultimi (l'uscita può sceglierne la
      variante)
```

2.3 Classi e strumenti Unreal

- `FGSTN_WallAnimSettings`, `FGSTN_WallAnimState`, `FGSTN_WallAnimInputs`, enum `EGSTN_WallAnimPhase` e `EGSTN_WallSlideAnimState` (`WallMovement/Animation/GSTN_WallAnimTypes.h`).
- `UGSTN_WallAnimUtils` (`UBlueprintFunctionLibrary`, senza stato): `SplitWallVelocity`, `ClassifySlide`, `StepWallAnimState`, `GatherInputs` (legge `UMoverComponent::GetMovementMode`, `GetVelocity`, `GetUpDirection` e `FGSTN_WallMovementSyncState::WallNormal`).
- `UGSTN_WallAnimInstance` (`UAnimInstance`): genitore C++ del layer della wall slide. In `NativeUpdateAnimation` aggiorna `WallAnim`; il grafo lo legge con *Property Access* (thread safe).
- Test `GSTN.Wall.Anim.*`: scomposizione con gravità diverse, classificazione con isteresi, passi di fase (entrata, slide→climb senza uscita, ripresa durante l'uscita, durate a zero).

2.4 Il grafo del layer (con animazioni segnaposto)

```
ABP_GSTN_WallLayer (genitore GSTN_WallAnimInstance, implementa ALI_GSTN_Wall), layer
    WallLayer:
  Enter --(Phase == Active)--> Loop
  Loop = macchina interna su SlideState: Idle / SlideLeft / SlideRight / Skid
        (sequenze o blend space su LateralAlpha, VerticalAlpha)
  Loop o Enter --(Phase == Exit)--> Exit
  Exit --(Phase == Enter)--> Enter (ripresa del muro durante l'uscita)
```

3 Animazioni per funzionalità, disaccoppiate dall'ABP

3.1 Ragionamento

Hai chiesto che wall slide, spin, trapano e simili portino le proprie animazioni senza legarle all'ABP, e che una skin (o qualche sua caratteristica) possa cambiarle lasciando identica la hitbox. È lo schema di Lyra: ogni funzionalità ha un *linked anim layer* (un anim Blueprint a parte, con la sua interfaccia), collegato con `LinkAnimClassLayers`; una variante è un anim Blueprint figlio. In più la scelta avviene per tag, per priorità tra le sorgenti.

3.2 Parte A (fatta)

- `IGSTN_OverrideLayerSource`: un layer dice se vuole prendere la posa (`IsOverrideLayerActive`), con quale tempo di blend e con che priorità.
- `UGSTN_CharacterAnimInstance`: genitore C++ dell'ABP principale. Non conosce nessuna funzionalità: a ogni frame chiede ai layer collegati (`GetLinkedAnimInstances`) chi è attivo ed espone `bOverrideLayerActive` e `OverrideBlendTime`. I layer collegati si aggiornano prima dell'istanza principale, quindi la risposta è del frame corrente.
- Nell'ABP principale servono solo: il reparent, un *Blend Poses by bool* e un nodo *Linked Anim Layer* per ogni interfaccia di layer. Nessuna modifica per ogni nuova variante.

3.3 Parte B (fatta): il plugin GSTNAnimation

```
plugin GSTNAnimation, modulo GSTNAnimationCore (puro: Engine, GameplayTags)
UGSTN_AnimSetDefinition (data asset): SetTag (Anim.Set.Wall/Spin/Drill), LayerClass,
                                  MontageOverrides { Reference, Visual }
FGSTN_AnimSetRule: Set, When (query sui tag cosmetici: SkinTags + Anim.Trait.*),
    Priority
UGSTN_AnimSetCollection: Rules (su personaggio, kit, arma, skin)
UGSTN_AnimSetComponent (solo presentazione):
  a ogni cambio di sorgente o di tag (mai a ogni frame):
    per ogni SetTag: vincitore = max(priorità sorgente, priorità regola) tra le regole
        che combaciano
    carica in asincrono, LinkAnimClassLayers(LayerClass), mappa Reference -> Visual
  ResolveVisualMontage(Reference)
```

3.4 La hitbox non cambia: le regole

1. La simulazione (timeline, finestre di hit e hurtbox, `BakedSocket`, mosse in coda di Mover) legge solo il montage *di riferimento* e non dipende dal componente degli anim set.
2. I socket si fanno bake solo dal montage di riferimento; una variante visiva non si fa mai bake.
3. Un validatore dell'editor richiede che ogni variante abbia le stesse sezioni, le stesse durate (entro un frame), lo stesso play rate e nessun root motion diverso.
4. Le hurtbox seguono l'`HurtboxSet` del kit, non la skin.

4 Le tue scelte e gli esempi

Hai scelto le **regole a tag**: le skin restano invariate e le loro caratteristiche sono tag. Con la scelta vanno il plugin a parte e la regola delle stesse sezioni e durate. I tuoi esempi, maschile/femminile e pesante/esile, sono i tratti d'esempio del plugin (`Config/Tags/GSTNAnimation.ini`): `Anim.Trait.Body.Male`, `Anim.Trait.Body.Female`, `Anim.Trait.Build.Heavy`, `Anim.Trait.Build.Slim`. Le regole si combinano:

```
collezione della skin:
  Anim.Set.Wall -> DA_AnimSet_Wall_Heavy quando Heavy
  Anim.Set.Wall -> DA_AnimSet_Wall_HeavyFemale quando Heavy E Female (priorità 1: più
      specifica)
skin pesante maschile -> Heavy
skin pesante femminile -> HeavyFemale
skin esile -> la wall slide base del personaggio
```

Una variante che cambia solo i montage (nessun layer) lascia collegato il layer della sorgente sotto: per esempio una skin femminile che cambia solo lo spin tiene il layer della wall slide del personaggio.

4.1 Classi del plugin

- `UGSTN_AnimSetDefinition`, `FGSTN_MontageOverride`, `FGSTN_AnimSetRule`, `UGSTN_AnimSetCollection` (`GSTN_AnimSetTypes.h`); la validazione dei dati controlla sezioni, durate, play rate e root motion delle varianti.
- `UGSTN_AnimSetUtils`: `ResolveWinners`, `ResolveLayerClasses`, `ValidateMontageOverride`.
- `UGSTN_AnimSetComponent`: sorgenti, tag cosmetici, caricamento asincrono con `FStreamableManager` in due passi, `LinkAnimClassLayers` / `UnlinkAnimClassLayers`, ricollegamento su `OnAnimInitialized` quando una skin cambia l'`AnimClass`, `ResolveVisualMontage`.
- Tag nativi `Anim`, `Anim.Set`, `Anim.Trait`; `Anim.Set.Wall` è nel modulo del progetto.
- Test `GSTNAnimation.Core.*`: priorità delle sorgenti e delle regole, scelta per tratti (anche combinati), layer di ripiego.

Da fare dopo: collegare kit, arma e skin al componente nel punto dove oggi si applica il loadout (B3).

5 Cosa controllare a mano

1. **Compilazione**: build completa (nuovo plugin `GSTNAnimation` abilitato nel `.uproject`), poi Session Frontend `GSTN.Wall.Anim` (3 test) e `GSTNAnimation.Core` (3 test).
2. **Editor**, quando è libero (Blueprint condiviso, faccio io o tu):
   (a) creare `ALI_GSTN_Wall` con il layer `WallLayer`;
   (b) creare `ABP_GSTN_WallLayer` (genitore `GSTN_WallAnimInstance`) con il grafo della sezione 2.4 e clip GASP segnaposto;
   (c) fare il reparent di `SandboxCharacter_Mover_ABP` a `GSTN_CharacterAnimInstance`, aggiungere il blend e il nodo `WallLayer`;
   (d) creare `DA_AnimSet_Wall` (`Anim.Set.Wall`, layer `ABP_GSTN_WallLayer`) e una collezione del personaggio che lo usa, poi aggiungere `GSTN_AnimSetComponent` a `SandboxCharacter_Mover` con quella collezione;
   (e) (prova delle varianti) un layer figlio con clip diverse, un set `Anim.Set.Wall` con quel layer e una regola `Anim.Trait.Build.Heavy`; aggiungere il tratto ai `DefaultCosmeticTags` del componente e vedere il layer cambiare.
3. **PIE**, mappa `L_WallSlideAB`, con `showdebug animation` o il debugger dell'ABP sul layer:
   (a) salto contro il muro da fermo: Enter, poi Active/Idle;

(b) stick a destra e a sinistra: SlideRight / SlideLeft, nessuno sfarfallio vicino a 80–120 cm/s;

(c) entrata in corsa veloce: Skid, che finisce sotto 650 cm/s;

(d) passaggio su una superficie da climb: resta Active (nessuna uscita), `bIsClimbing` vero, niente skid;

(e) salto dal muro o atterraggio: Exit con `ExitModeName` giusto, poi None;

(f) in listen server, un client che guarda un altro giocatore: stessi stati (lo stato viene dalla presentazione smussata).